

Parallel Shark Smell Optimization

Pietro Cipriani

000000

pietro.cipriani@studenti.unitn.it

Filippo De Grandi

257824

filippo.degrandi@studenti.unitn.it

Giacomo Vettore

000000

giacomo.vettore@studenti.unitn.it

Abstract—The Shark Smell Optimization (SSO) algorithm is a metaheuristic optimization technique inspired by the hunting behavior of sharks. Due to the high computational cost of population-based optimization, this paper presents various parallel implementations of SSO using MPI, OpenMP and hybrid MPI+OpenMP programming models. The MPI version distributes the population across processes, while the hybrid approach combines distributed-memory and shared-memory parallelism to improve resource utilization on multicore systems. Performance evaluation on standard benchmark functions analyzes execution time, speedup, and scalability for different population sizes and processor configurations.

Three decomposition strategies are investigated: shark-level (population), dimension-level, and rotation-level parallelism. Performance evaluation on the Rastrigin benchmark function with a standard configuration ($np=1000$, $nd=200$, $k_max=1000$, $m=50$) shows that shark-level decomposition achieves near-linear speedup in both MPI and OpenMP, reaching $43\times$ with 64 MPI processes and $50\times$ with 64 OpenMP threads. Dimension-level and rotation-level strategies exhibit poor scaling because their work granularity is too small to amortize synchronization and communication costs. The hybrid MPI+OpenMP shark-level implementation achieves the shortest wall-clock time, reaching $450\times$ over serial at $p=64$, $t=32$ (2048 total cores, 22% efficiency).

Index Terms—parallelization, hpc, mpi, openmp, sharks, smelling

I. INTRODUCTION

Meta-heuristic optimization algorithms have become a widespread tool for solving complex nonlinear optimization problems where deterministic or gradient-based approaches are computationally infeasible¹. Population-based optimization methods are effective in exploring high-dimensional search spaces and handling multimodal objective functions. These techniques are widely applied in engineering optimization, machine learning, scheduling, computational biology, and scientific computing.

Although the Shark Smell Optimization (SSO) algorithm demonstrates strong optimization capabilities on nonlinear benchmark functions, its iterative population-based structure introduces substantial computational costs as the number of sharks, problem dimensionality, and iteration count increase. Parallelization becomes essential to reduce execution time and improve scalability.

This paper investigates multiple parallel implementations of the Shark Smell Optimization algorithm using Message

Passing Interface (MPI), OpenMP, and hybrid MPI+OpenMP programming models. In addition to comparing distributed-memory and shared-memory paradigms, the study analyzes different parallelization strategies within the algorithm itself. Specifically, parallel execution is applied at three distinct levels:

- shark-level parallelism, where candidate solutions are distributed across population elements;
- dimension-level parallelism, where computations across search-space dimensions are parallelized;
- hybrid parallelism, combining both the other strategies.

By combining these algorithmic decomposition strategies with MPI, OpenMP, and hybrid MPI+OpenMP execution models, the work evaluates multiple parallel variants of SSO and analyzes their scalability, synchronization behavior, communication overhead, and computational efficiency on multicore HPC architectures.

A. The Shark Smell Optimization Algorithm

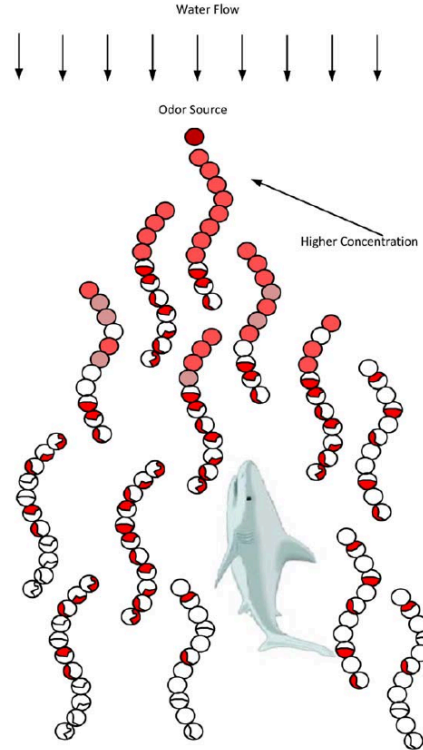


Fig. 1. Schematic illustration of a shark's movement to odor source.

¹We found this “bestiary” with a particular hot-take on meta-heuristic algorithms [1]

The SSO [2] algorithm is a population-based metaheuristic inspired by sharks' ability to locate prey by sensing and following odor gradients.

The method maintains a population of candidate solutions (sharks) that move through the search space combining global exploration and focused local exploitation.

```
1 struct Shark {
2     double *position; // Current position of the shark.
3     double *speed;    // Current speed of the shark.
4     double pos_score; // The OF(position) value.
5 };
```

Listing 1. Shark data structure

Each shark iteratively updates its position and speed based on a combination of inertia (momentum from previous movement) and attraction toward promising regions of the search space (global best or local neighborhood best). Additionally, a local rotational search is performed around improved sharks to refine their positions.

The main stages of the SSO algorithm are described in the following pseudo-code, which highlights the key computational steps:

```
Serial Algorithm
1 // Shark loop
2 for shark_index in 0 .. cfg.np-1:
3     shark = sharks[shark_index]
4     // Stages loop
5     for k in 0 .. cfg.k_max-1:
6         R1 = uniform(0,1)
7         R2 = uniform(0,1)
8
9         // 1) per-dimension derivative and speed update
10        for dim in 0 .. cfg.nd-1:
11            derivative = eval_derivative(shark.position,
12                                         cfg.nd, cfg.obj, dim)
13            grad_term = cfg.eta * R1 * derivative
14            mom_term = cfg.alpha * R2 * shark.speed[dim]
15            shark.speed[dim] = grad_term + mom_term
16            // apply velocity limiter using cfg.beta
17            if abs(shark.speed[dim]) > abs(cfg.beta *
18                shark.speed[dim]):
19                shark.speed[dim] = cfg.beta * shark.speed[dim]
20
21        // 2) forward movement and clamp
22        for dim in 0 .. cfg.nd-1:
23            shark.position[dim] = clamp(shark.position[dim]
24                                       + shark.speed[dim], domain[dim])
25
26        // 3) rotational local search
27        best = OF(shark.position, cfg.nd, cfg.obj)
28        best_r3 = 0
29        for m in 0 .. cfg.rotations-1:
30            r3 = uniform(-1,1)
31            for dim in 0 .. cfg.nd-1:
32                candidate[dim] = clamp(shark.position[dim] *
33                                       (1 + r3), domain[dim])
34            val = OF(candidate, cfg.nd, cfg.obj)
```

```
31     if val > best: best = val; best_r3 = r3
32     if best_r3 != 0:
33         for dim in 0 .. cfg.nd-1:
34             shark.position[dim] =
35                 clamp(shark.position[dim] * (1 + best_r3),
36                     domain[dim])
37
38     // 4) update global best
39     cur_min = -best
40     if cur_min < best_min:
41         best_min = cur_min
42     copy(best_pos, shark.position)
43
44 // result in best_min, best_pos
```

a) *Design intuition:*

- Inertia preserves momentum and enables smooth exploration of the search space.
- Attraction terms bias motion toward high-quality regions (global best or local neighborhood best).
- Rotational local search acts as an exploitation operator similar to a focused local search or mutation that samples around a candidate on a small hypervolume to refine its position.

II. PARALLEL DESIGN

The SSO algorithm contains various sources of parallelism. The evaluation and update of individual sharks can generally be executed independently, making shark-level decomposition highly suitable for distributed-memory execution. Conversely, dimension-level decomposition is a more complicated form of parallelism that can be applied within the evaluation of a single shark's position and speed updates, which may be more efficient in shared-memory environments. Hybrid approaches can combine these strategies to leverage both inter-node and intra-node parallelism.

In the following graph, we illustrate the slowdown of the serial algorithm as various parameters (number of sharks, dimensions, rotations, iterations) increase.

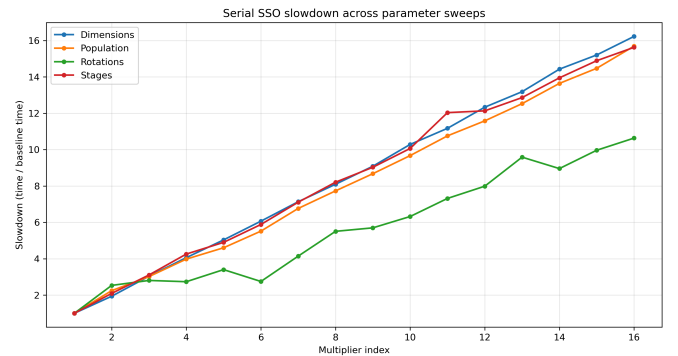


Fig. 2. Execution time of the serial SSO algorithm as a function of the number of sharks, dimensions, stages and rotations.

The graph above underlines how the SSO algorithm would benefit from parallelization, especially as the problem size increases. The computational cost grows significantly with

the number of sharks, dimensions, and iterations, making it crucial to explore parallel design strategies to achieve practical execution times for larger problem instances.

Although the outer “stage” loop over the stage index k , in the graph indicated by the red line, might at first seem parallelizable, it is inherently sequential: each iteration at stage k consumes and modifies data produced at stage $k-1$ (for example updated shark positions, velocities and fitness values). These true data dependencies prevent concurrent execution of different k iterations without violating correctness. Any approach to overlap would require complex checkpointing or synchronization that typically outweighs potential gains. Therefore, parallelism is applied at the shark and dimension levels rather than across stage iterations.

This work therefore investigates the interaction between:

- the programming model:
 - MPI,
 - OpenMP,
 - hybrid MPI+OpenMP;
- and the algorithmic decomposition strategy:
 - shark-level parallelization,
 - dimension-level parallelization,
 - rotation-level parallelization,
 - hybrid parallelization.

The comparative evaluation of these configurations enables a detailed analysis of scalability, workload distribution, communication overhead, synchronization costs, and overall parallel efficiency.

A. Shark-level Parallelism

The shark population is the most natural decomposition unit in SSO. Each shark evolves independently across all stages; no inter-shark communication occurs during the update loop. Assigning disjoint subsets of sharks to different workers therefore yields an embarrassingly parallel workload. The only required global coordination is a single reduction at the end of the run to identify the globally best solution. The communication-to-computation ratio decreases proportionally as the population grows, making shark-level decomposition strongly scalable.

A practical design consideration is PRNG independence. Each shark requires its own random stream for velocity updates (R1, R2) and rotational probes (R3). In the MPI implementation, ranks use rank-offset seeds; in the OpenMP implementation, each thread carries a thread-local seed. This avoids both contention on a shared RNG and correlation artifacts between workers.

The principal limitation of this strategy is granularity: if the population is small relative to the number of workers (e.g., $n_p = 1000$ with 1024 processes), some workers receive zero or one shark, wasting resources.

B. Dimension-level Parallelism

Within the velocity and position update of a single shark (Eq. 7–9 of the original algorithm), computations across different dimensions are independent: the partial derivative of

the objective function in dimension j depends only on the current position vector, not on other dimensions’ derivatives. This permits fine-grained n_d -way parallelism inside the inner loops of the SSO update.

However, several structural constraints limit the practical effectiveness of this approach:

- **Granularity:** the maximum useful worker count equals n_d (200 in the standard configuration), and each work unit is a single gradient evaluation, which is a very lightweight operation.
- **Synchronization frequency:** before the rotational search can start, all dimension updates must complete. This creates $O(k_{\max} \times n_p)$ synchronization barriers per run, which amounts to $O(10^6)$ barriers with the standard parameters.
- **Communication volume (MPI):** the complete position vector must be reconstructed on every rank after each speed update via `MPI_Allgatherv`, incurring $O(n_p \times n_d)$ bytes of communication per stage.

Dimension-level decomposition is beneficial only when the objective function evaluation is expensive and n_d is large enough to amortize the per-barrier overhead.

C. Rotation-level Parallelism

The M rotational probes in the local search (Eq. 10) are mutually independent: each probe draws a scalar $r3$ uniformly at random and evaluates the objective at a scaled position around the current shark. The best probe is then selected with a simple maximum reduction over M values.

Like shark-level decomposition, this loop is embarrassingly parallel. Unlike it, the available parallelism is capped by M rather than by the population: with $M = 50$ and 64 workers, each worker receives fewer than one probe on average, making the parallel overhead dominate. Rotation-level parallelism therefore becomes effective only when M is substantially larger than the number of workers, which corresponds to configurations where a fine-grained local search is required for solution quality. In our experiments we also tested an extended configuration ($M = 50000$) to expose the scaling behavior of this strategy under more favorable conditions.

III. IMPLEMENTATION

All code and scripts related to the project are available in the repository (see `README`). The implementation reuses a single algorithmic core, found in `sso/sso.c` and exposes multiple execution strategies through self-contained entrypoints in the `src` directory, e.g. `src/main_mpi_sharks.c`, `src/main_openmp_dim.c`, `src/main_hybrid_dim.c`, etc. Each entrypoint implements a specific parallelization strategy and programming model, while sharing the same underlying SSO logic.

The names of the entrypoints follow the convention `main_<model>_<strategy>.c`, where `<model>` is one of `mpi`, `openmp`, or `hybrid`, and `<strategy>` indicates the parallelization approach, i.e. `sharks`, `dim`, `rot`.

The following subsections describe the main parallelization strategies we implemented, the key code locations, and the design trade-offs.

A. MPI Implementations

We implemented three MPI decompositions that map naturally to the SSO algorithm:

- **Shark-level (population) decomposition:** each MPI rank owns a subset of the population and executes the full per-shark stages locally. At the end of the run ranks perform a reduction to determine the globally best solution. See `src/main_mpi_sharks.c`. This approach is straightforward and minimizes communication during the main loop, as each rank can independently update its sharks. This results in low communication overhead and good scalability when the population size is large enough to amortize the cost of setup and evaluation. However, it may suffer from load imbalance if the sharks are not uniformly distributed in terms of computational cost (e.g. some sharks may converge faster than others).
- **Dimension-level decomposition:** the decision-variable vector is partitioned across MPI ranks; each rank computes updates for its set of dimensions and `MPI_Allgatherv` is used to reconstruct full position vectors when needed. See `src/main_mpi_dim.c`. Here, the main loop is parallelized across dimensions, which can be beneficial when the cost of evaluating the objective function or computing gradients is dominated by per-dimension work (e.g. very high `nd`). However, this approach introduces higher communication overhead due to the need for frequent all-gather operations to share updated positions across ranks, and it requires more complex synchronization to ensure consistency of the shared state.
- **Rotation-level decomposition:** the inner rotational local-search probes are distributed across ranks and the best candidate is found with a reduction (e.g. `MPI_MAXLOC` or custom two-value reduction). See `src/main_mpi_rot.c`. Like the dimension-level approach, this strategy can be effective when the cost of the rotational search is significant (e.g. large rotations), but it also introduces communication overhead due to the need for reductions to find the best candidate across ranks. However, it minimizes redundant objective evaluations since each rank only evaluates a subset of the rotations, and the reduction combines results efficiently.

For all MPI variants we chose straightforward collective patterns (`Allreduce`, `Allgatherv`, `Bcast`) to keep the code readable and portable.

B. OpenMP Implementations

OpenMP variants exploit shared memory to avoid message passing; the code, like the MPI variants, uses different decomposition strategies depending on which loop is most expensive.

- **Shark-level (thread over sharks):** each OpenMP thread processes different sharks end-to-end and maintains thread-local scratch buffers and per-thread best values to reduce contention. See `src/main_openmp_sharks.c`. A parallel region is created through `#pragma omp parallel` and the outer loop over sharks is partitioned with `#pragma omp for`. Each thread maintains its own RNG state and

scratch arrays to avoid data races and false sharing. At the end of the loop, a reduction is performed to find the global best solution across threads.

- **Dimension-level (thread over dimensions):** A single `#pragma omp parallel` region is opened outside all loops; the thread team then processes the outer shark and stage loops collaboratively. Within each stage, the velocity update and position update loops are parallelized with `#pragma omp for schedule(static)`, while the random scalars `R1` and `R2` are drawn inside a `#pragma omp single` block to guarantee all threads use a consistent pair. The rotational search is also guarded by `#pragma omp single`, so it runs on one thread while the others wait at the implicit barrier. See `src/main_openmp_dim.c`.
- **Rotation-level (parallel rotation probes):** The outer shark and stage loops remain serial; the inner rotation loop is parallelized with `#pragma omp for schedule(static)` inside a parallel region re-entered at every stage. A custom OpenMP reduction over a `RotationBest` struct (declared with `#pragma omp declare reduction`) finds the best (`r3`, `value`) pair across threads without a critical section. Each thread holds its own candidate buffer and per-thread seed (derived from `seed_base + tid < 16`) to avoid false sharing and PRNG contention. See `src/main_openmp_rot.c`.

OpenMP versions emphasize minimizing synchronization points and using per-thread storage for temporary arrays.

C. Hybrid MPI+OpenMP Implementations

The hybrid implementation (`src/main_hybrid_sharks.c`) combines MPI shark-level decomposition with intra-rank OpenMP parallelism. Each MPI rank owns a contiguous block of the population and evolves its local sharks through all stages. Within each rank, OpenMP threads parallelize the rotational local search of each shark using the same `rot_best` reduction pattern as the OpenMP rotation variant. This two-level hierarchy maps naturally to multi-node, multi-core clusters: MPI handles coarse-grained distribution across nodes, while OpenMP exploits the shared memory within each node without message-passing overhead.

Key design choices in the hybrid implementation:

- Each rank seeds its local PRNG with `seed_base + rank` to ensure statistically independent random streams across nodes.
- Each OpenMP thread uses a further per-thread seed `seed_base + rank + (tid < 16)` to avoid intra-rank correlations.
- A single `MPI_Allreduce` with a custom two-value struct reduction at the end combines local best values into the global best, minimizing synchronization to one collective call per run.
- The PBS script (`tests/hybrid_sharks`) uses `--map-by ppr:N:node:PE=T` together with `OMP_PLACES=cores` and `OMP_PROC_BIND=close` to pin OpenMP threads to physically adjacent cores within each socket, reducing NUMA overhead.

The total worker count is $P \times T$ where P is the number of MPI processes and T is the number of OpenMP threads per process. For a fixed total of $P \times T$ workers, the hybrid configuration typically outperforms the pure-MPI or pure-OpenMP equivalents by reducing the number of expensive MPI_Allreduce calls (fewer ranks) while still exploiting all available cores.

D. Notes and Supporting Scripts

The algorithmic core (`sso_update_speed`, `sso_move_forward`, `sso_unrotational_search`, `sso_sharks_alloc`) is implemented in `src/sso/sso.c` and reused across every main entrypoint; this keeps the implementations comparable and reduces duplication.

Argument parsing and default parameters live in `src/sso/parse_args.c` and are extended in each `main_*` file to add parallel-specific options (`threads`, `dim-procs`, `tsharks`, etc.). Benchmarking and plotting tools are under `benchmarks/` and `tests/` (examples: `tests/launch_tests.sh`, `benchmarks/plot_results.py`). These scripts were used to generate the figures reported in Section 4.

IV. PERFORMANCE AND SCALABILITY ANALYSIS

In this section, we present a performance analysis of the various parallel implementations of the SSO algorithm. We evaluate execution time, speedup and efficiency across the different parallelization strategies and programming models. The experiments are conducted on a multicore HPC cluster, and we analyze both strong and weak scalability. The section is organized as follows:

- Experimental setup and benchmark functions
- Analysis with varying number of processes
- Analysis with varying number of threads
- Analysis with both processes and threads (hybrid)
- Analysis with different execution modes in the PBS system.

A. Experimental Setup and Benchmark Functions

All experiments use a fixed parameter set chosen to produce runs long enough for reliable timing while remaining tractable on the cluster:

- Number of sharks (`np`): 1000
- Number of dimensions (`nd`): 200
- Number of stages (`k_max`): 1000
- Number of rotations (`rotations`): 50

The remaining algorithm parameters (`eta`, `alpha`, `beta`) are set to the values from the original SSO paper [2].

A set of scripts was developed to automate job submission across different process and thread counts. The scripts generate PBS job files from templates and throttle submission to stay within the queue's concurrent-job limit. All jobs run in `excl` placement mode to prevent resource sharing with other workloads and ensure consistent wall-clock measurements.

```
1  ...
2  for procs in "${PROC_VALUES[@]"; do
3    for thrds in "${THRD_VALUES[@]"; do
4      while [[ $(qstat -u $USER | wc -l) -ge 30 ]]; do
5        sleep 10
6      done
7      sed \
8        -e "s/\${N_THRD}/$thrds/g" \
9        -e "s/\${N_PROCS}/$procs/g" \
10       -e "s/\${JOB_NAME}/$JOB_NAME/g" \
11       -e "s/\${N_CPUS}/$N_CPUS/g" \
12       -e "s/\${PLACE}/$PLACE/g" \
13       "$EXEC" | qsub
14    done
15  done
```

Listing 2. A simplified portion of the script used to launch the tests on the HPC cluster. The script iterates over different values of processes and threads (user-defined), checks for the number of running jobs, and submits new jobs using `qsub` with the appropriate parameters.

```
1  #!/bin/env bash
2  #PBS -l
3  select=${N_CPUS}:ncpus=${N_PROCS}:mpiprocs=${N_PROCS}:mem
4  #PBS -l place=${PLACE}:excl
5  #PBS -l walltime=0:5:00
6  #PBS -q shortCPUQ
7
8  # output & error files
9  #PBS -N ${JOB_NAME}_${N_PROCS}_${N_THRD}
10
11  module load OpenMPI/4.1.6-GCC-13.2.0
12
13  mpirun -n (( ${N_CPUS} * ${N_PROCS} )) ./parallel-sso/
14  sso_hybrid_sharks -p 1000 -d 200 -k 1000 -m 50 -t
15  ${N_THRD}
```

Listing 3. An example PBS script, edited by Listing 2. The script specifies resource requirements, loads necessary modules, and runs the executable with the defined parameters for sharks, dimensions, stages, rotations, and threads.

The implementation contains three different objective functions (in `sso/ofuncs.c`), taken from the original SSO paper, that are used for testing the algorithm's performance and scalability. In our experiments, we used the Rastrigin function [3] (set as default in the code), as the differences between the various objective functions were negligible in terms of execution time and speedup.

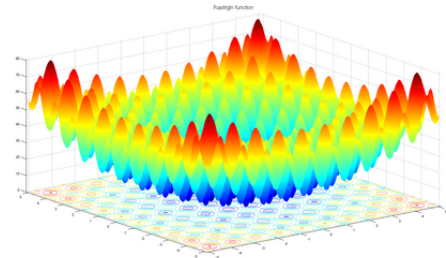


Fig. 3. Rastrigin function, a common benchmark for optimization algorithms.

B. Varying number of processes

Fig. 4, Fig. 5, and Fig. 6 show execution time for the three MPI decomposition strategies with 1 to 64 processes. The OpenMP results appear in the same plots to allow direct comparison; their analysis follows in the next subsection.

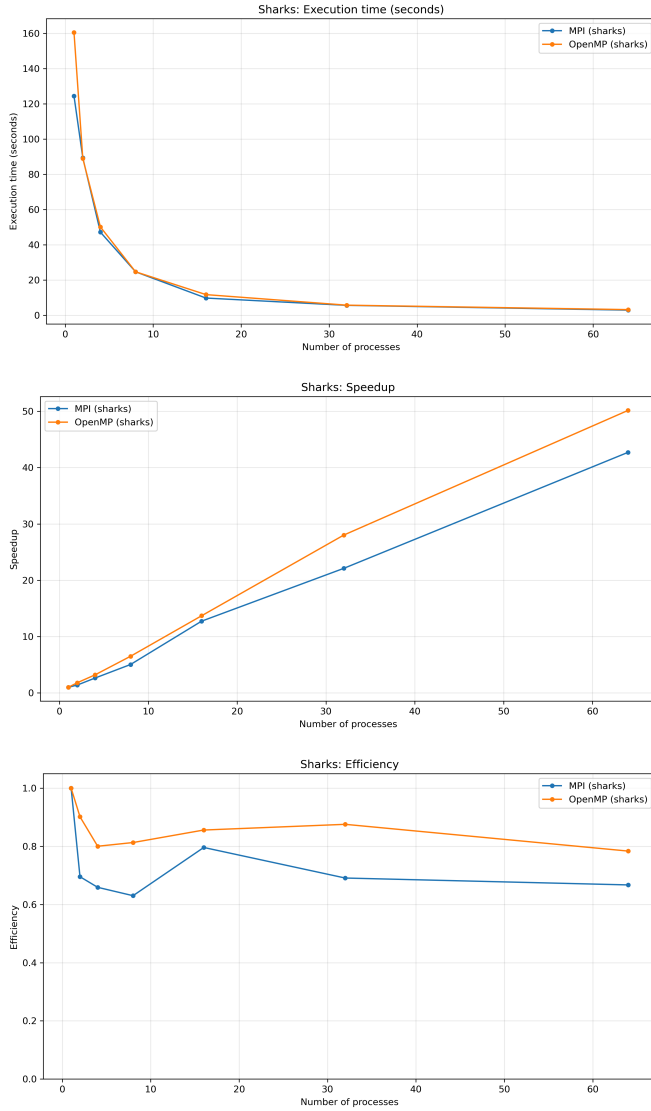


Fig. 4. Shark-level decomposition: execution time, speedup, and efficiency for MPI (processes) and OpenMP (threads).

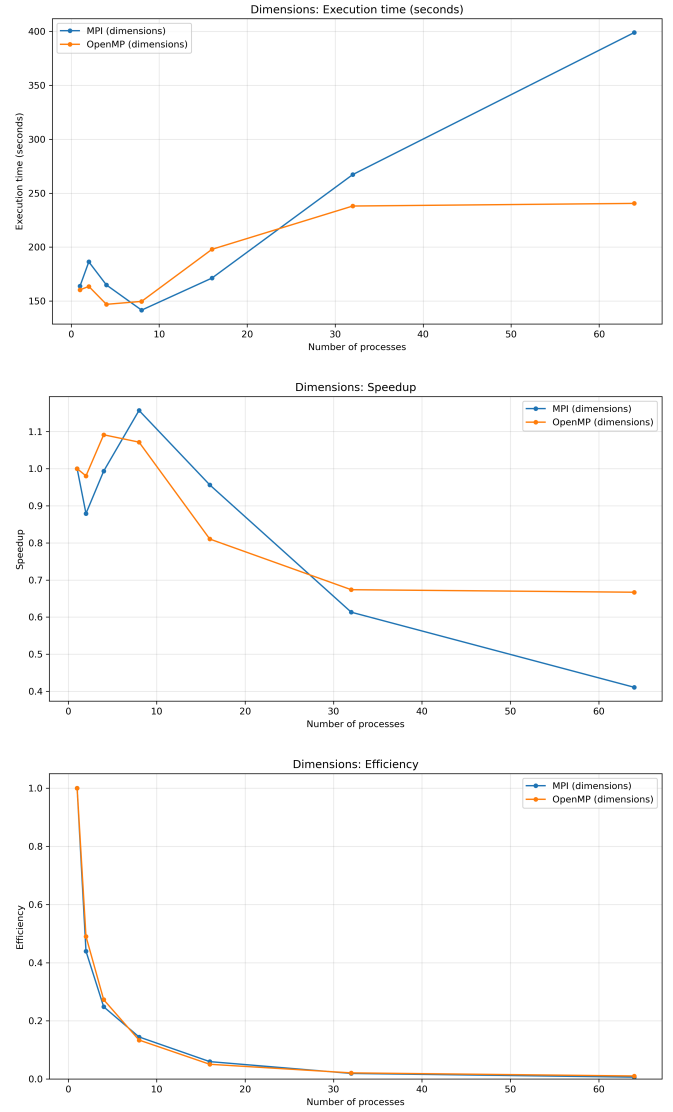


Fig. 5. Dimension-level decomposition: execution time, speedup, and efficiency for MPI (processes) and OpenMP (threads).

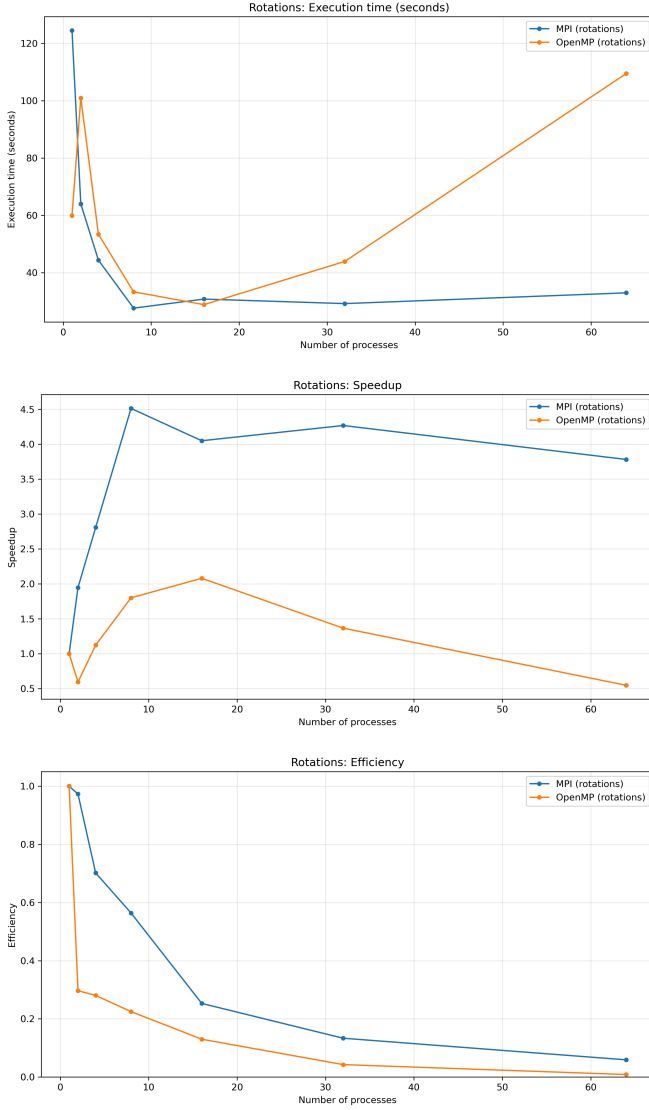


Fig. 6. Rotation-level decomposition: execution time, speedup, and efficiency for MPI (processes) and OpenMP (threads). Standard configuration: $m=50$.

Shark-level MPI. The single-process baseline is approximately 124 s. Execution time halves at every doubling up to 16 processes (speedup 12.7 $\times$) and continues to improve to 2.9 s at 64 processes (speedup 42.7 $\times$, efficiency 67%). The mild super-linear behaviour visible at 8–16 processes is attributable to each rank’s local population fitting more readily into cache. The efficiency decline at 64 processes reflects the growing relative cost of the final MPI_Allreduce.

Dimension-level MPI. This variant shows no useful speedup. The single-process time of approximately 164 s rises to 399 s at 64 processes, which is more than 2.4 $\times$ slower. The root cause is the MPI_Allgather call required to reassemble the full position vector after each speed update: with $np=1000$ sharks and $k_{\max}=1000$ stages the run issues roughly 10^6 all-gather calls, each transferring 200 doubles. At high process counts the all-gather latency and bandwidth demand dominate the computation entirely.

Rotation-level MPI. With only 50 rotations per shark per stage, speedup saturates quickly. At 2 processes the speedup is 1.95 $\times$ (near-ideal); at 8 processes it reaches 4.5 $\times$ (efficiency 56%); beyond 8 processes performance degrades because each rank evaluates fewer than 7 probes per stage and the synchronization cost of the two-value MPI_Allreduce outweighs the saved computation.

To verify that rotation-level parallelism can scale when given sufficient work, a second set of experiments used a substantially larger rotation count. Results appear in Fig. 7.

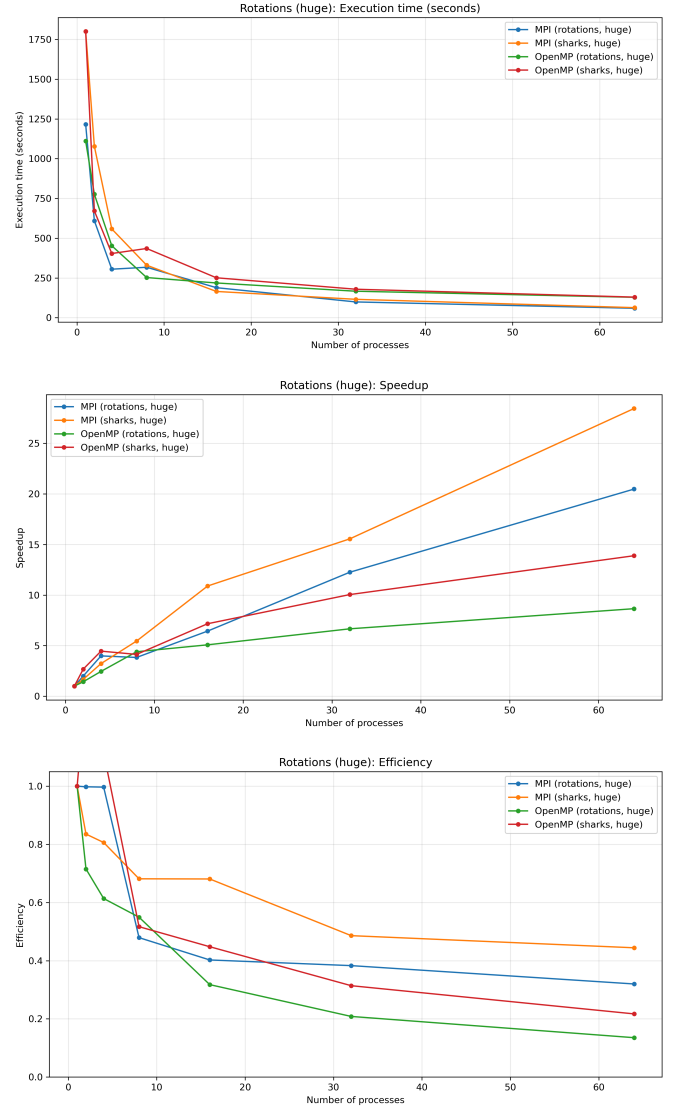


Fig. 7. Rotation-level and shark-level decomposition with an extended workload: execution time, speedup, and efficiency. The `mpi_sharks_huge p=1` data point is a 5-minute walltime timeout; its true serial time is larger than 1800 s.

With the extended rotation count, the MPI rotation variant reaches a speedup of approximately 20 $\times$ at 64 processes. This confirms that the poor scaling in the standard configuration stems from communication overhead relative to compute, not from a fundamental algorithmic barrier.

C. Varying number of threads

The OpenMP results are included in the same figures as their MPI counterparts (Fig. 4, Fig. 5, Fig. 6) to allow direct comparison.

Shark-level OpenMP. Results closely follow the MPI shark variant. The single-thread baseline is approximately 160 s (slightly higher than the MPI $p=1$ baseline because OpenMP thread-team initialisation is charged to the timed region). At 64 threads the time drops to 3.2 s, giving a speedup of $50\times$ and efficiency of 78%. The shared-memory model eliminates all inter-process communication; the only synchronization is a single `#pragma omp critical` block at the end of the parallel region to fold per-thread best values into the global best. OpenMP therefore achieves higher efficiency than MPI at the same worker count, as the communication term is entirely absent.

Dimension-level OpenMP. Performance is essentially flat or worse across all thread counts. The single-thread time is approximately 160 s; at 64 threads it reaches approximately 241 s, a slowdown of $1.5\times$. Each `#pragma omp for` loop carries an implicit barrier at its end, generating one barrier per dimension-update pass, per shark, per stage. With the standard parameters this amounts to $2 \times n_p \times k_{\max} = 2 \times 10^6$ barriers per run, each with negligibly small work inside. The synchronization overhead dominates entirely, confirming that dimension-level parallelism requires a much heavier per-dimension kernel to be viable.

Rotation-level OpenMP. The standard ($m=50$) variant shows erratic behaviour. At 1–4 threads there is a slight speedup, but at 32–64 threads performance is worse than single-threaded. The reason is that the `#pragma omp parallel` region is re-entered once per shark per stage, resulting in $n_p \times k_{\max} = 10^6$ parallel-region entries per run. Each entry carries thread-synchronization overhead even when the thread pool is reused. With only 50 iterations to distribute, the overhead-to-work ratio is unfavourable at any thread count beyond a handful. With the extended rotation count (Fig. 7) the picture improves markedly: the OpenMP variant reaches approximately $8.7\times$ at 64 threads, with efficiency still low but consistently improving.

D. Hybrid parallelism

Fig. 8 shows execution time, speedup and efficiency for the hybrid MPI+OpenMP shark-level implementation as a function of the number of threads per process, with the number of MPI processes varied from 1 to 64.

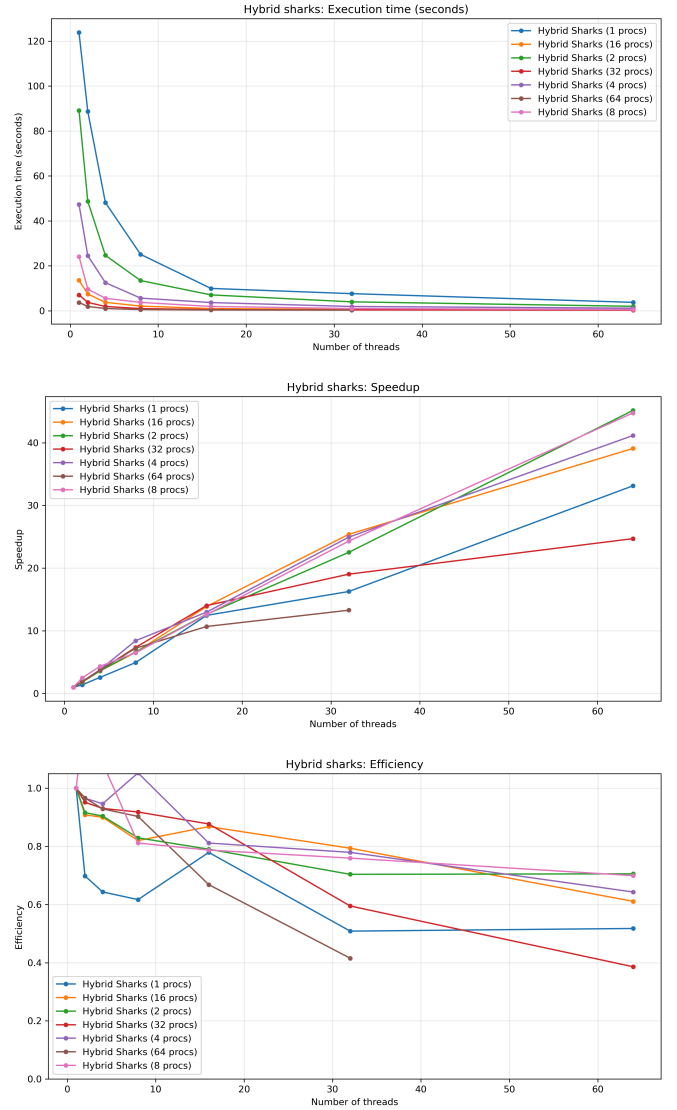


Fig. 8. Hybrid MPI+OpenMP shark-level implementation: execution time, speedup, and efficiency. Each line corresponds to a fixed number of MPI processes; the x-axis is the number of OpenMP threads per process.

For a fixed number of MPI processes, adding OpenMP threads reduces execution time near-linearly up to roughly 16–32 threads per process, after which gains diminish. With $p=8$ processes and $t=64$ threads (512 total cores), execution time drops to 0.54 s, giving a speedup of approximately $230\times$ at 45% efficiency. The best raw result is $p=64$, $t=32$ (2048 total cores) at 0.277 s, which is $450\times$ faster than the serial baseline of 124.6 s. That corresponds to a parallel efficiency of 22%, which reflects the thin per-rank population (roughly 16 sharks per MPI process) at that configuration.

Comparing hybrid to pure-MPI at the same total worker count $P \times T$ reveals a consistent advantage for the hybrid configuration at high total counts. Eight MPI processes with 8 threads each (64 total workers) complete in 3.71 s (speedup $33.4\times$, efficiency 52%), while pure MPI at 64 processes completes in 2.91 s (speedup $42.7\times$, efficiency 67%). Pure MPI wins at 64 total workers because the inter-process reduction is cheap for a small reduction vector. However, as the total

worker count grows further, the hybrid approach scales better: with $p=16$ and $t=64$ (1024 total workers) the time is 0.35 s, a regime where pure MPI at 1024 processes would face severe load imbalance since each rank would hold fewer than one shark.

Efficiency drops below 30% at very high combined counts ($p=32$, $t=32$ or $p=64$, $t=16$) because the local population per rank falls to 1–2 sharks and any imbalance in per-shark computation time is magnified. The practical operating point for this problem size is around $p=8$ –16 processes and $t=8$ –32 threads, yielding speedups of 50–230 $\times$ with efficiencies of 30–52%.

E. Playing with PBS

All experiments were executed on the HPC cluster using the PBS scheduler in `excl` mode to prevent resource sharing with other jobs and ensure reproducible wall-clock measurements. Jobs were submitted via the scripting infrastructure described in Listing 2 and Listing 3.

Placement policy. The PBS `place` directive controls how the allocated chunks (nodes) are mapped to the physical topology. We experimented with two main policies:

- **scatter:** each chunk is allocated on a distinct physical node. This is the natural choice for MPI-heavy workloads because it maximizes the aggregate memory bandwidth across the job, and each MPI rank gets exclusive access to a full node’s cores and memory.
- **pack:** chunks are packed as tightly as possible, sharing nodes. This policy minimizes inter-node communication latency (useful for the hybrid case where MPI ranks within the same node communicate via shared memory rather than the network fabric).

For the pure MPI shark and rotation tests we used `scatter` to ensure predictable core counts per rank. For the hybrid tests we used `scatter` as well, because the `--map-by ppr:N:node:PE=T` option already controls intra-node placement, and scattering nodes prevents inadvertent sharing.

Walltime and queue. All jobs were submitted to the `shortCPUQ` queue. Sharks-level tests used a 5-minute walltime; rotation and dimension tests used 15 minutes because single-process runs are longer. The `mpi_sharks_huge` experiment at $p=1$ hit the 5-minute wall limit and was recorded as 1800 s (visible in Fig. 7). The true single-process time for that configuration is therefore larger than 1800 s, which means the reported speedup values for that variant are conservative lower bounds.

V. CONCLUSION

This paper studied nine parallel variants of the Shark Smell Optimization algorithm, crossing three decomposition strategies (shark-level, dimension-level, rotation-level) with three programming models (MPI, OpenMP, hybrid MPI+OpenMP).

The central finding is that the decomposition strategy matters far more than the programming model. Shark-level decomposition is embarrassingly parallel and scales almost linearly with worker count in both MPI and OpenMP, reaching

43 $\times$ at 64 MPI processes and 50 $\times$ at 64 OpenMP threads on the Rastrigin benchmark. Dimension-level and rotation-level decompositions both fail to scale under the standard parameter set because their work granularity (a single gradient evaluation or a single rotation probe) is too small to amortize synchronization and communication costs. With a larger rotation count, rotation-level MPI recovers and reaches 20 $\times$ at 64 processes, confirming that these strategies are viable only when the per-unit workload is sufficient.

The hybrid MPI+OpenMP shark-level variant combines the two scalable axes and achieves the shortest wall-clock time: 0.277 s at $p=64$, $t=32$ (2048 total cores), a 450 $\times$ reduction over the serial baseline at 22% efficiency. The practical operating point for the standard problem size is $p=8$ –16 processes and $t=8$ –32 threads, which yields 50 $\times$ –230 $\times$ speedup at 30–52% efficiency with far fewer resources.

Two limitations apply broadly. First, population-level load balance relies on uniform per-shark cost; if the objective function evaluation becomes problem-dependent and variable, work stealing or dynamic scheduling would be needed. Second, none of the variants exploit GPU acceleration, which would be particularly natural for the embarrassingly parallel rotation search and could extend the speedup by another order of magnitude.

REFERENCES

- [1] A. Campelo, “EC Bestiary: A bestiary of evolutionary, swarm and other metaphor-based algorithms,” *Zenovo*, [Online]. Available: <https://fcampelo.github.io/EC-Bestiary/>
- [2] O. Abedinia, N. Amjady, and A. Ghasemi, “A new metaheuristic algorithm based on shark smell optimization,” *Complexity*, vol. 21, no. 5, pp. 97–116, 2016, doi: <https://doi.org/10.1002/cplx.21634>.
- [3] Wikipedia, “Rastrigin function,” [Online]. Available: https://en.wikipedia.org/wiki/Rastrigin_function